

LA ROCHELLE UNIVERSITÉ

LICENSE 3 INFORMATIQUE



Computer Vision & Artificial Intelligence

Project Report: Chess Game Video Analysis

Professors MASCARILLA Laurent
PETERI Renaud

Members TRAN Minh Duong

Abstract

This project involves developing a Python application using OpenCV to automatically analyze and transcribe chess game from a top-down video stream. The system is divided into several computer vision tasks: detecting and rectifying the chessboard, tracking player hand movements to determine when a move has been completed, and identifying the position and color of chess pieces. The final output is a full transcription of the game in algebraic notation.

Introduction

Chess is a popular strategy game that demands rigorous logical analysis. In the context of computer vision, automating the transcription of a physical game presents a complex challenge: transforming a continuous 3D video stream into digital data.

By using a top-down camera feed and OpenCV, this project aims to bridge the gap between the physical board and digital notation. By leveraging image processing techniques such as perspective transformation, motion analysis,... we can create an autonomous system capable of “reading” the game in real-time. This automation eliminates human error in recording moves and allows for immediate digital analysis using algebraic notation.

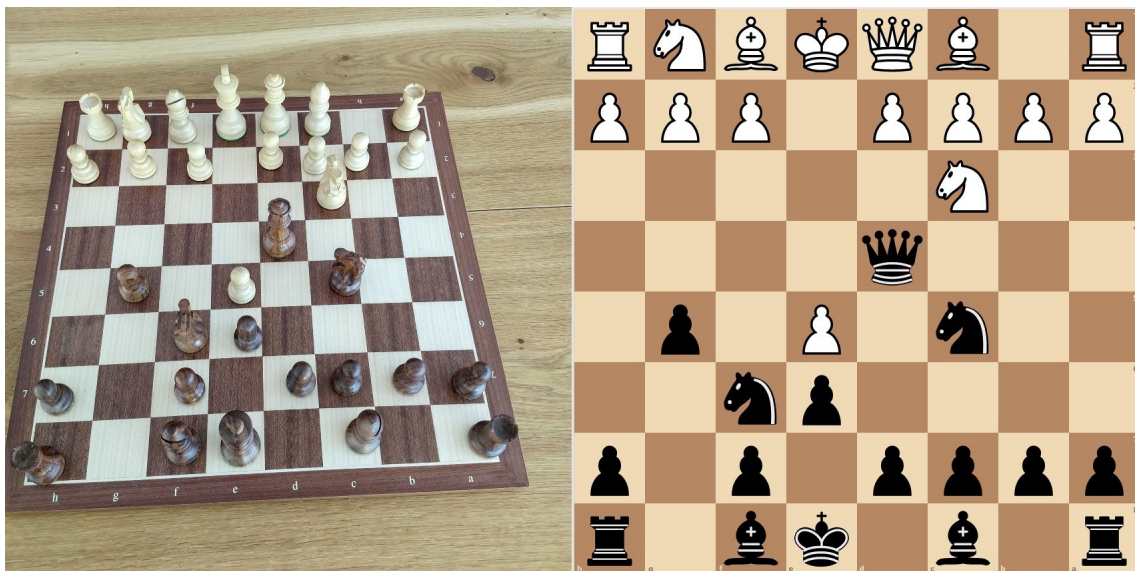


Figure 1: Converting a physical chessboard into a digital one

I. Board Detection and Rectification

I.1. Objective

The primary goal of this part is to extract the 8×8 chessboard from the raw video feed, then transform it into a top-down square image. This helps eliminate perspective distortion and background noise, such as the table, tripod or player movements, which could otherwise interfere with the analysis.

I.2. Methodology

The pipeline for board rectification is divided into three main processes:

A. Image Preprocessing - Edges Detection

To extract the board structure, the frame is first converted to grayscale and smoothed using a **Gaussian Blur** with a 3×3 kernel to filter out high-frequency noise. We then apply a **Canny Edge Detector** to identify the grid boundaries. Finally, a **Morphological Closure** (3×3 rect kernel) is performed to fill small gaps in the grid lines, ensuring a continuous contour regardless of lighting conditions.

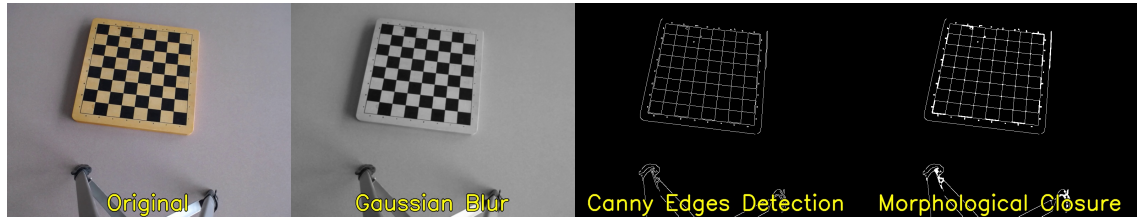


Figure 2: Preprocessing Workflow: from the raw input to the closed-edge map

B. Corner Extraction

The system identifies the chessboard by finding the largest external contour in the edge map, assuming the 8×8 grid is the most suitable geometric feature. To extract the four specific corners from this contour, we use the following coordinate extremes:

- **Top-Left:** Point with the minimum $(x + y)$ sum.
- **Bottom-Right:** Point with the maximum $(x + y)$ sum.
- **Top-Right:** Point with the minimum $(y - x)$ difference.
- **Bottom-Left:** Point with the maximum $(y - x)$ difference.

C. Perspective Transformation

Once the corners are localized, a **Homography Matrix** is calculated using OpenCV's `getPerspectiveTransform`. This matrix maps the distorted coordinates to a perfect 400×400 pixel square. We then apply `warpPerspective` to generate the rectified “top-down” view ready for game transcription.

I.3. Results and Discussion

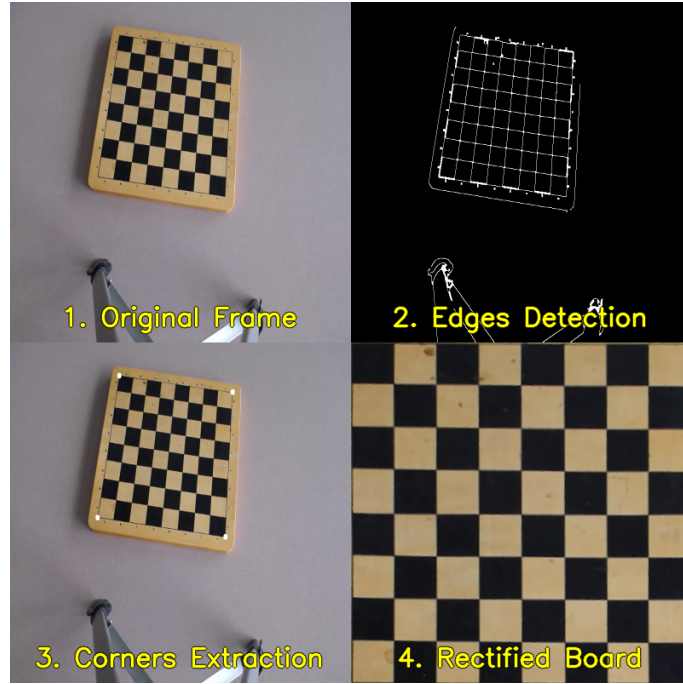


Figure 3: Board Extraction Result

The implementation successfully generates a clean, standardized view of the board. By extracting these corner coordinates from an empty-board video and storing them into `corners_coordinates.json`, we have essentially “**locked**” the chessboard so that the system can remain stable throughout the game.

While the rectified image simplifies determining cell occupancy, it is noted that the vertical shape of pieces may be distorted. Consequently, while the rectified board is used for coordinate mapping, the *original frame* may be referenced for piece identification to preserve visual features.